线上健身房会员约课管理系统建模例题

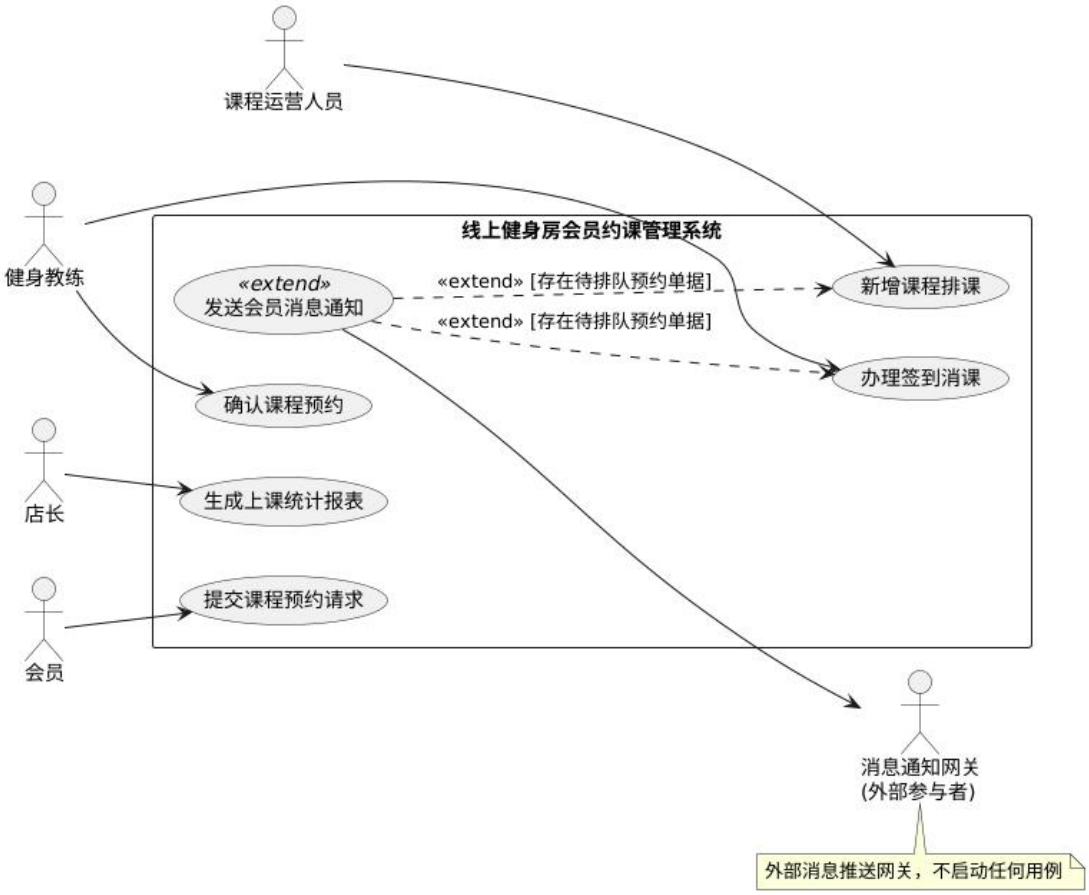
业务需求如下：

开发线上健身房会员约课管理系统，提升健身房课程预约业务效率。会员提交课程预约请求（单次仅预约 1 节课程），系统查询课程剩余可预约名额；若课程有名额，生成预约记录，扣减课程可预约名额；健身教练执行确认上课预约操作；若课程名额已满，生成排队预约记录与排队单。健身教练办理消课签到：会员上完课完成签到消课，恢复课程名额；签到消课后若存在该课程待排队的排队单，则排队单转为待确认状态，调用外部消息通知服务通知排队会员，仍需要教练确认预约。课程运营人员新增排课，新课排课成功后更新课程名额；系统查看是否存在该课程的会员排队单，如果有排队，则把排队单转为待确认状态，调用外部通知服务通知会员，教练后续确认预约。系统根据已完成消课记录生成会员上课统计报表提供给店长查看。

请根据需求描述，建立用例模型，并创建类图、顺序图，状态图等分析模型。

一、用例模型

参与者	类型	说明
会员	主参与者	发起课程预约、排队请求，单次预约 1 节课程
健身教练	主参与者	确认课程预约、办理会员签到消课
课程运营人员	主参与者	新增课程排课，查看排队记录
店长	主参与者	查看会员上课统计报表
消息通知网关 MessageNotifyGateway	次要外部参与者	外部短信 / 消息推送系统；系统在排课新增、签到消课触发排队激活时调用，不启动任何用例



UC01 提交课程预约请求

用例 ID：UC01 用例名称：提交课程预约请求 参与者：会员（主参与者） 输入参数：memberInfo 会员信息、courseId 课程编号、bookCount 预约节数

约束：单次预约仅支持 1 节课程，bookCount=1

前置条件：会员登录系统，拥有正常约课权限

后置条件

分支 A (课程名额充足可约)：生成待确认预约单，课程可预约名额扣减，生成待确认单据供教练确认预约。

分支 B (课程名额已满)：生成待排队预约单 + 排队记录，课程名额不变；排队记录通过单据编号关联预约单据获取会员信息，排队数量等于预约申请节数 bookCount=1。

基本事件流

1. 会员提交课程预约请求（课程编号、预约节数）
2. 系统查询课程名额记录
3. 系统判断课程名额充足可约
4. 系统生成预约单据记录
5. 系统更新课程名额（扣减可预约名额），用例结束

备选事件流

3a：课程名额不足已满。系统生成排队记录，绑定单据编号、课程编号、queueNum 排队节数；生成待排队预约单；课程名额不修改，用例结束。

说明：排队记录不重复存储会员信息，通过 orderId 关联 CourseOrder 单据获取会员信息。

UC02 确认课程预约

用例 ID：UC02

用例名称：确认课程预约

参与者：健身教练（主参与者）

输入参数：orderId 单据编号

前置条件：系统存在状态为 PENDING_CONFIRM 待确认的预约单据

后置条件：对应单据标记为 BOOKED 已预约

基本事件流

1. 健身教练查看系统待确认单据列表
2. 教练确认会员课程预约
3. 教练确认操作，修改单据状态为【BOOKED 已预约】，用例结束

备选事件流

1a：orderId 不存在：系统提示单据不存在，用例结束。

1b：单据存在，但状态不是 PENDING_CONFIRM 待确认：系统提示单据状态不允许确认预约，用例结束。

UC03 新增课程排课

用例 ID：UC03

用例名称：新增课程排课

参与者：课程运营人员（主参与者）

输入参数：courseId 课程编号、addNum 新增可预约名额

前置条件：课程运营人员拥有排课操作权限，系统可访问课程名额数据

后置条件：课程名额更新；系统自动检索对应课程排队单据；若存在待排队预约单据，**按预约提交时间先后顺序激活排队单据**，触发扩展用例【发送会员消息通知】；排队单据转为待确认，对应排队记录更新为 HANDLED 已处理。

基本事件流

1. 课程运营人员查询查看系统排队记录
2. 运营人员完成课程新增排课
3. 系统更新课程名额记录（增加课程可预约名额）
4. 系统检索该课程对应的会员排队单据，**按预约提交时间先后筛选待排队单据**
5. 如果存在待排队预约单据： 5.1 更新排队单据状态为 PENDING_CONFIRM 待确认 5.2 更新排队记录状态 HANDLED 已处理 5.3 触发扩展用例：发送会员消息通知，用例结束

备选事件流

4a：无对应待排队预约单据：仅更新课程名额，不做单据状态变更，不触发扩展用例，用例结束。

UC_EXT01 发送会员消息通知【扩展用例规约】

用例 ID：UC_EXT01

用例名称：发送会员消息通知

参与者：无主参与者，被系统内部调用

扩展点：新增课程排课、办理签到消课；

触发条件：存在待排队预约单据

前置条件：排队单据已经更新为待确认，存在待通知的会员列表（从 CourseOrder 关联获取会员信息）

后置条件：调用外部消息通知网关发送通知；发送失败记录系统日志，不改变任何业务实体状态，不回滚业务数据。

基本事件流

1. 业务主用例传入待通知会员信息列表
2. MessageNotifyUI 组装通知消息
3. 调用外部消息通知网关 MessageNotifyGateway 发送通知
4. 获取通知返回结果；成功则用例结束。

备选事件流

3a：通知发送失败。系统记录错误日志；不回滚预约单、排队记录、课程名额状态；扩展用例结束。

UC04 生成上课统计报表

用例 ID：UC04

用例名称：生成上课统计报表

参与者：店长（主参与者）

输入参数：timeRange 统计时间区间

前置条件：系统保存有预约单据记录

后置条件：仅基于状态为 COMPLETED 已完成消课单据生成上课统计报表，支持查看、导出；系统支持定时器后台自动生成报表。

基本事件流

1. 店长传入统计时间区间，触发生成上课统计报表
2. 系统读取该时间区间内状态为 COMPLETED 已完成消课单据
3. 系统汇总数据生成上课统计报表对象
4. 报表展示给店长，用例结束

备选事件流

3a：无已完成消课记录：系统提示暂无上课数据，报表为空，用例结束。

UC05 办理签到消课

用例 ID：UC05

用例名称：办理签到消课

参与者：健身教练（主参与者）

输入参数：orderId 单据编号

约束：当前系统单次签到消课仅支持处理 1 节课程，一张预约单据对应 1 节课程。

前置条件：健身教练拥有签到消课操作权限

后置条件：预约单据更新为 COMPLETED 已完成；课程可预约名额 +1；若存在待排队预约单，**排队单据按照预约提交时间先后激活**，排队单转为待确认，排队记录标记已处理，触发扩展用例【发送会员消息通知】。

基本事件流

1. 健身教练录入单据编号 orderId，发起签到消课操作
2. 系统查询该单据下状态为 BOOKED 已预约单据
3. 教练确认会员到场上课（1 节）
4. 系统修改预约单据状态为 COMPLETED 已完成
5. 系统更新课程名额，课程可预约名额 +1
6. 系统查询该课程下 PENDING_QUEUE 待排队预约单据，按预约提交时间先后筛选
7. 如果存在待排队预约单据：

    7.1 将排队单据状态更新 PENDING_CONFIRM 待确认

    7.2 更新排队记录 HANDLED 已处理

    7.3 触发扩展用例：发送会员消息通知，用例结束

备选事件流

3a：找不到已预约单据：系统提示无该笔预约记录，用例结束。

7a：不存在待排队预约单据：仅更新课程名额和消课单据，不触发扩展用例，用例结束。

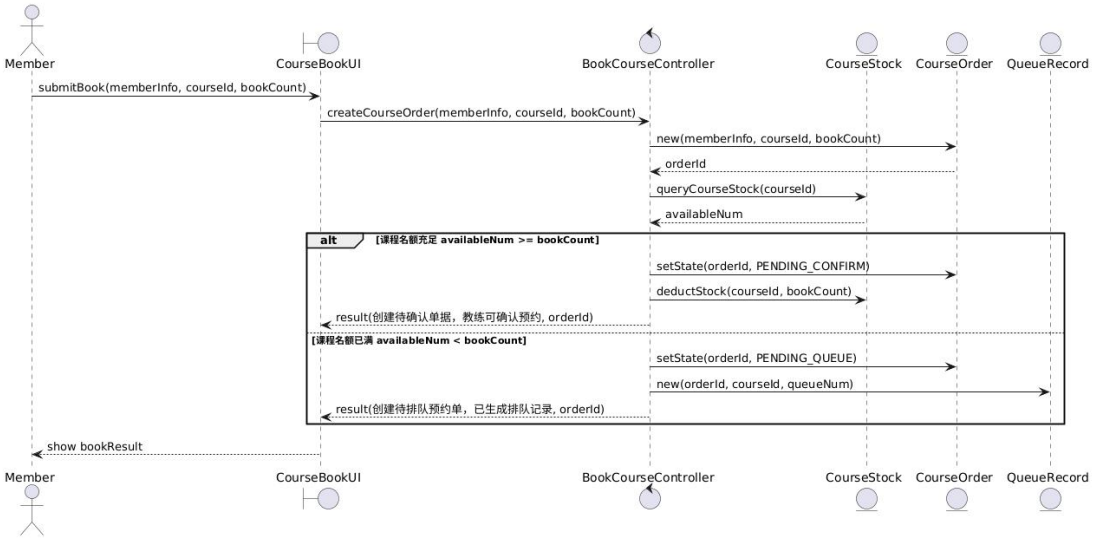
二、识别概念类

分类	类名	说明
边界类	CourseBookUI	会员课程预约界面
边界类	CoachUI	健身教练操作界面
边界类	SignUI	签到消课操作界面，教练录入签到信息
边界类	ScheduleUI	课程运营人员排课操作界面
边界类	ReportViewUI	店长报表查看界面
边界类	MessageNotifyUI	消息通知边界类；组装通知消息，调用外部通知网关
边界类	MessageNotifyGateway	外部消息通知网关接口，对接第三方消息服务，对应次要外部参与者
控制类	BookCourseController	会员课程预约业务控制
控制类	ConfirmBookController	教练确认预约业务控制
控制类	SignController	签到消课业务控制，消课、名额更新、排队激活
控制类	ScheduleController	课程排课、排队单据状态更新控制
控制类	CourseReportController	上课统计报表业务控制

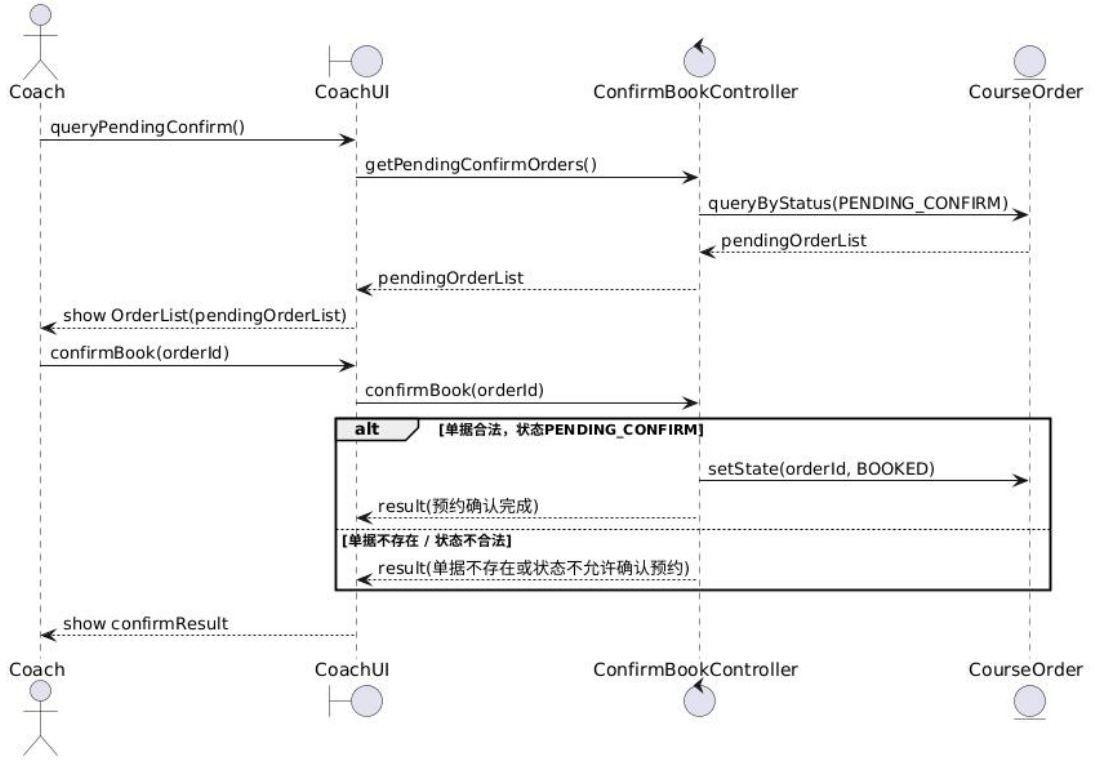
分类	类名	说明
实体类	CourseOrder	预约 / 排队单据；属性：单据编号、课程编号、会员、节数、预约提交时间、状态；状态：PENDING_CONFIRM 待确认、PENDING_QUEUE 待排队、BOOKED 已预约、COMPLETED 已完成
实体类	CourseStock	课程名额实体；属性：课程编号、可预约名额 availableNum
实体类	QueueRecord	排队记录；关联 CourseOrder 单据号、课程编号、排队节数 queueNum；通过 orderId 关联 CourseOrder 获取会员信息；状态 UNHANDLED 未处理、HANDLED 已处理；一张 CourseOrder 最多对应一条 QueueRecord
实体类	CourseStat	上课统计报表实体；存储统计结果对象

三、分析模型

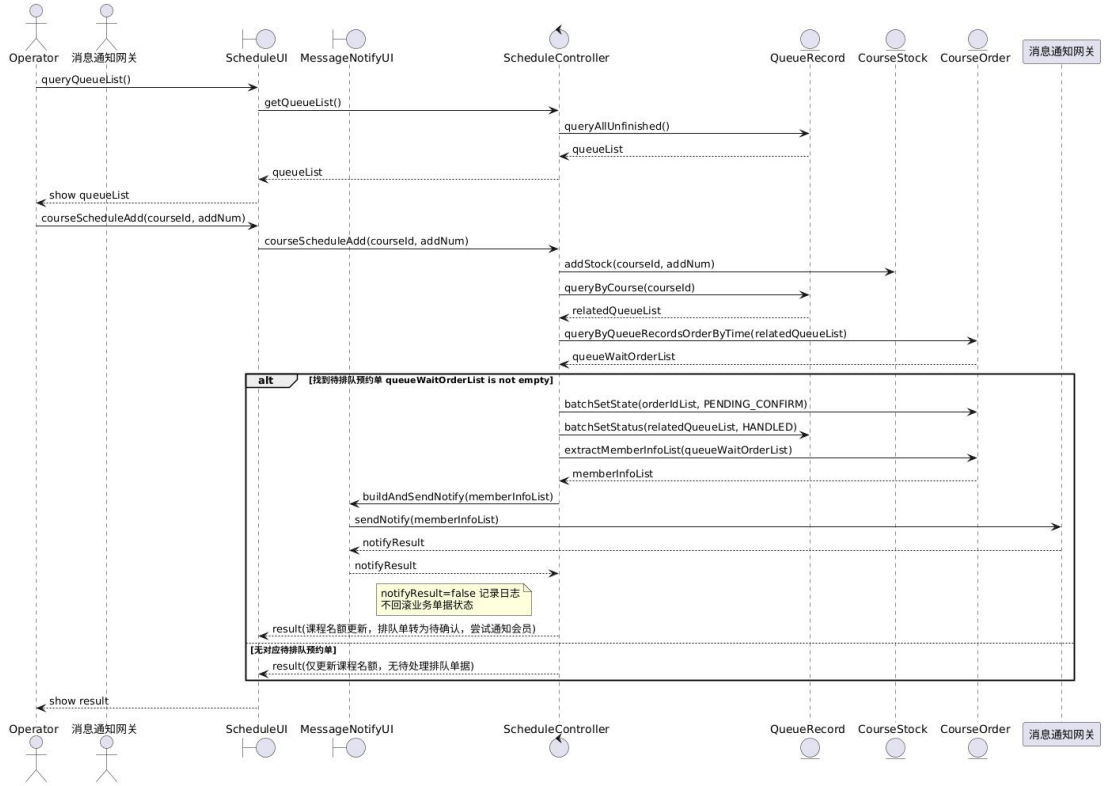
UC01 顺序图



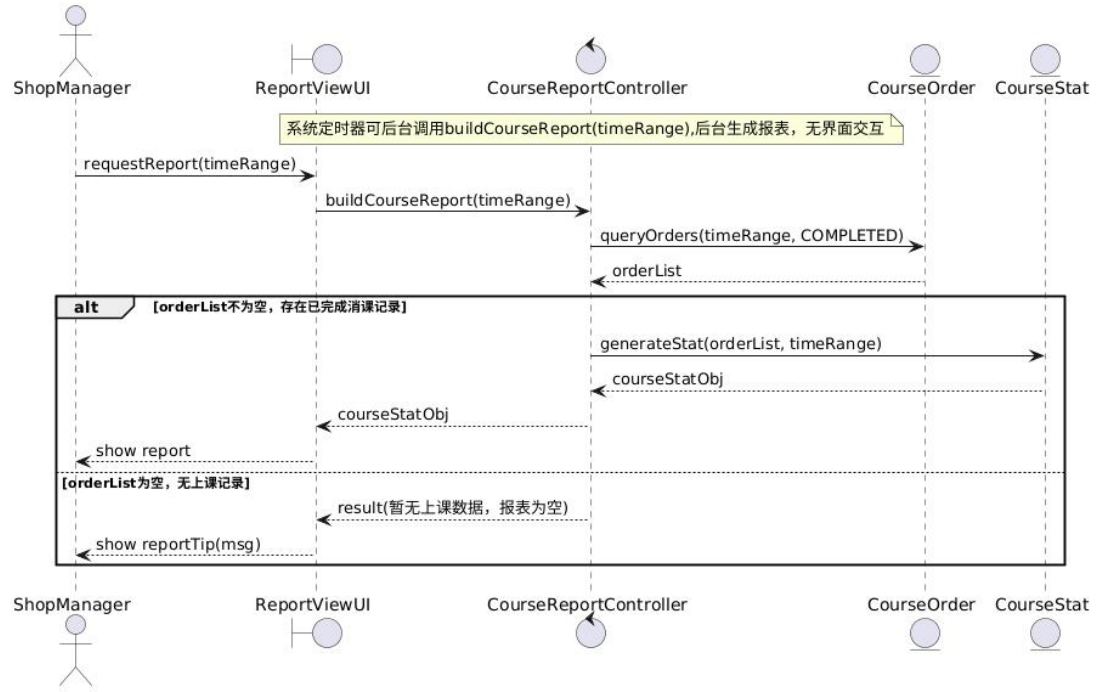
UC02 顺序图



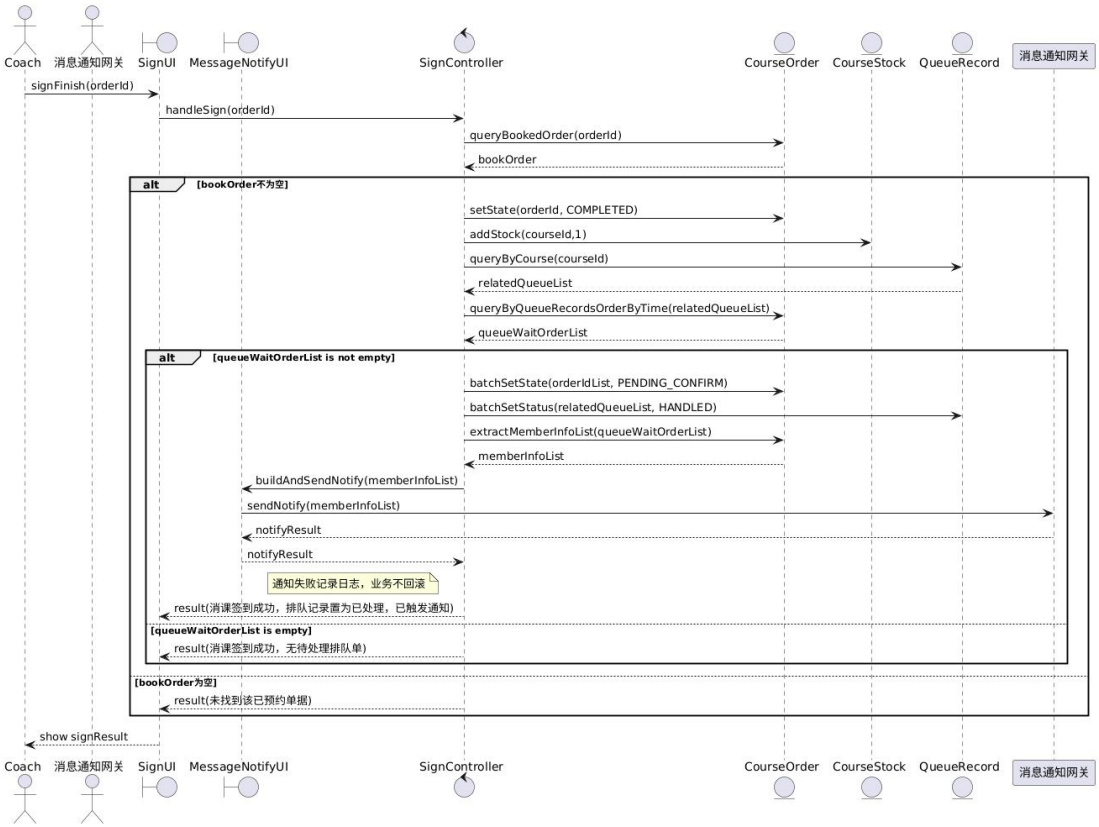
UC03 顺序图


Operator
消息通知网关
ScheduleUI
MessageNotifyUI
ScheduleController
QueueRecord
CourseStock
CourseOrder
消息通知网关
queryQueueList()
getQueueList()
queryAllUnfinished()
queueList
queueList
show queueList
courseScheduleAdd(courseId, addNum)
courseScheduleAdd(courseId, addNum)
addStock(courseId, addNum)
queryByCourse(courseId)
relatedQueueList
queryByQueueRecordsOrderByTime(relatedQueueList)
queueWaitOrderList
alt
[找到待排队预约单 queueWaitOrderList is not empty]
batchSetState(orderIdList, PENDING_CONFIRM)
batchSetStatus(relatedQueueList, HANDLED)
extractMemberInfoList(queueWaitOrderList)
memberInfoList
buildAndSendNotify(memberInfoList)
sendNotify(memberInfoList)
notifyResult
notifyResult
notifyResult=false 记录日志
不回滚业务单据状态
result(课程名额更新，排队单转为待确认，尝试通知会员)
[无对应待排队预约单]
result(仅更新课程名额，无待处理排队单据)
show result
Operator
消息通知网关
ScheduleUI
MessageNotifyUI
ScheduleController
QueueRecord
CourseStock
CourseOrder
消息通知网关

UC04 顺序图


ShopManager
ReportViewUI
CourseReportController
CourseOrder
CourseStat
系统定时器可后台调用buildCourseReport(timeRange),后台生成报表，无界面交互
requestReport(timeRange)
buildCourseReport(timeRange)
queryOrders(timeRange, COMPLETED)
orderList
alt
[orderList不为空，存在已完成消课记录]
generateStat(orderList, timeRange)
courseStatObj
courseStatObj
show report
[orderList为空，无上课记录]
result(暂无上课数据，报表为空)
show reportTip(msg)
ShopManager
ReportViewUI
CourseReportController
CourseOrder
CourseStat

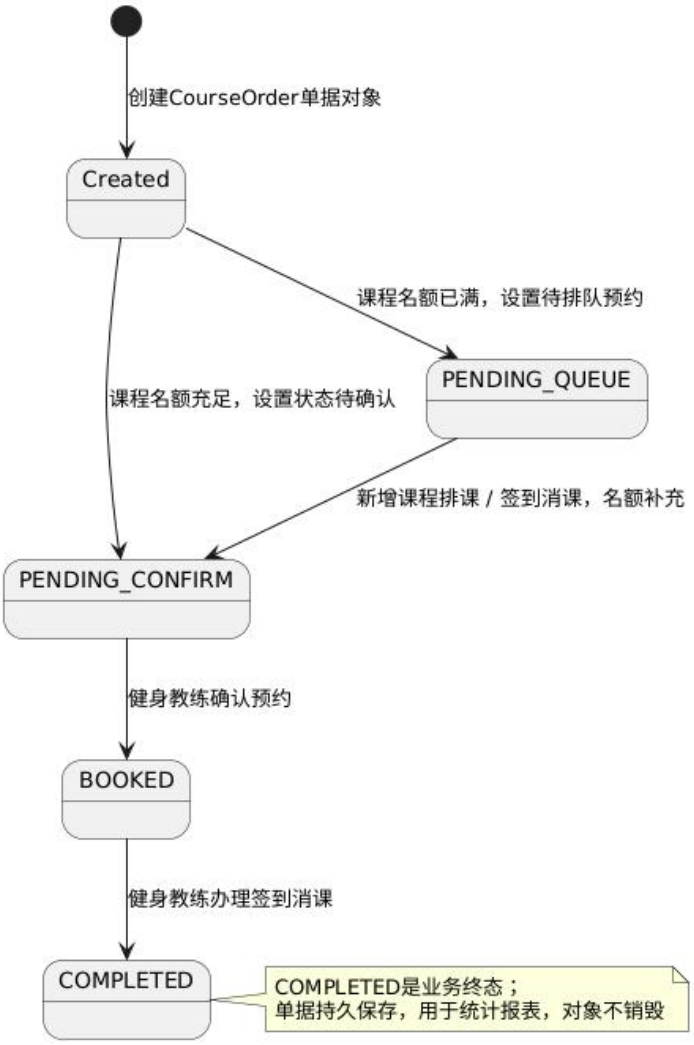
UC05 顺序图


Coach
消息通知网关
SignUI
MessageNotifyUI
SignController
CourseOrder
CourseStock
QueueRecord
消息通知网关
signFinish(orderId)
handleSign(orderId)
queryBookedOrder(orderId)
bookOrder
alt
[bookOrder不为空]
setState(orderId, COMPLETED)
addStock(courseId,1)
queryByCourse(courseId)
relatedQueueList
queryByQueueRecordsOrderByTime(relatedQueueList)
queueWaitOrderList
alt
[queueWaitOrderList is not empty]
batchSetState(orderIdList, PENDING_CONFIRM)
batchSetStatus(relatedQueueList, HANDLED)
extractMemberInfoList(queueWaitOrderList)
memberInfoList
buildAndSendNotify(memberInfoList)
sendNotify(memberInfoList)
notifyResult
notifyResult
通知失败记录日志，业务不回滚
result(消课签到成功，排队记录置为已处理，已触发通知)
[queueWaitOrderList is empty]
result(消课签到成功，无待处理排队单)
[bookOrder为空]
result(未找到该已预约单据)
show signResult
Coach
消息通知网关
SignUI
MessageNotifyUI
SignController
CourseOrder
CourseStock
QueueRecord
消息通知网关

类图


ScheduleUI
+ queryQueueList()
+ courseScheduleAdd(courseId:String, addNum:Integer)
SignUI
+ signFinish(orderId:String)
CoachUI
+ queryPendingConfirm()
+ confirmBook(orderId:String)
CourseBookUI
+ submitBook(memberInfo:String, courseId:String, bookCount:Integer)
ReportViewUI
+ requestReport(timeRange:String)
ScheduleController
+ getQueueList(): List<QueueRecord>
+ courseScheduleAdd(courseId:String, addNum:Integer):String
SignController
+ handleSign(orderId:String): String
ConfirmBookController
+ getPendingConfirmOrders(): List<CourseOrder>
+ confirmBook(orderId:String): String
BookCourseController
+ createCourseOrder(memberInfo:String, courseId:String, bookCount:Integer): String
CourseReportController
+ buildCourseReport(timeRange:String): CourseStat
CourseOrder
- orderId: String
- courseId: String
- memberInfo: String
- bookCount: Integer
- createTime: String
- status: String
// PENDING_CONFIRM / PENDING_QUEUE / BOOKED / COMPLETED
+ new(memberInfo:String, courseId:String, bookCount:Integer)
+ setState(orderId:String, status:String)
+ queryByStatus(status:String): List<CourseOrder>
+ queryBookedOrder(orderId:String): CourseOrder
+ queryByQueueRecordsOrderByTime(resList:List): List<CourseOrder>
+ batchSetState(idList:List, status:String)
+ extractMemberInfoList(orderList:List): List
CourseStock
- courseId: String
- availableNum: Integer
+ queryCourseStock(courseId:String): Integer
+ deductStock(courseId:String, count:Integer)
+ addStock(courseId:String, addNum:Integer)
MessageNotifyUI
+ buildAndSendNotify(memberInfoList:List): Boolean
CourseStat
- statId: String
- timeRange: String
- totalFinish: Integer
+ generateStat(orderSet:List, timeRange:String): CourseStat
1
0..1
QueueRecord
- queueId: String
- orderId: String
- courseId: String
- queueNum: Integer
- status: String // UNHANDLED未处理 / HANDLED已处理
+ new(orderId:String, courseId:String, queueNum:Integer)
+ queryAllUnfinished(): List<QueueRecord>
+ queryByCourse(courseId:String): List<QueueRecord>
+ batchSetStatus(resList:List, status:String)
MessageNotifyGateway
+ sendNotify(memberInfoList:List): Boolean

状态图：


创建CourseOrder单据对象
Created
课程名额已满，设置待排队预约
课程名额充足，设置状态待确认
PENDING_QUEUE
新增课程排课 / 签到消课，名额补充
PENDING_CONFIRM
健身教练确认预约
BOOKED
健身教练办理签到消课
COMPLETED
COMPLETED是业务终态；
单据持久保存，用于统计报表，对象不销毁

CRC 模型：

分类	类名	职责（知道什么 / 做什么）	协作者
边界类	**CourseBookUI** 会员预约界面	1. 接收会员提交的预约输入（memberInfo、courseId、bookCount）； 2. 把预约请求转发给业务控制对象； 3. 接收预约 / 排队处理结果并展示给会员	BookCourseController
边界类	**CoachUI** 教练操作界面	1. 查询待确认预约单据列表； 2. 接收教练确认预约操作（orderId）； 3. 转发请求、展示确认结果	ConfirmBookController
边界类	**SignUI** 签到消课界面	1. 接收教练签到消课输入（orderId）； 2. 转发签到消课请求； 3. 展示消课 / 排队激活结果	SignController
边界类	**ScheduleUI** 排课操作界面	1. 查询展示排队记录； 2. 接收新增排课输入（courseId、addNum）； 3. 转发排课请求、展示排课与排队激活结果	ScheduleController
边界类	**ReportViewUI** 店长报表界面	1. 接收统计时间区间输入； 2. 转发报表生成请求； 3. 展示报表或空数据提示	CourseReportController
边界类	**MessageNotifyUI** 消息通知组装	1. 接收待通知会员信息列表； 2. 组装通知消息； 3. 调用外部网关发送通知； 4. 发送失败记录日志	MessageNotifyGateway
边界类	**MessageNotifyGateway** 外部通知网关	1. 接收消息推送请求； 2. 调用第三方推送，返回成功 / 失败结果	**无**（被动被调用，不主动调任何人）

分类	类名	职责（知道什么 / 做什么）	协作者
控制类	**BookCourseController** 预约业务控制	1. 处理会员预约完整业务流程; 2. 创建 CourseOrder 单据对象; 3. 查询课程可预约名额; 4. 按名额充足 / 已满分支：充足→置 PENDING_CONFIRM 并扣减名额;已满→置 PENDING_QUEUE 并创建排队记录; 5. 返回处理结果	CourseOrder、CourseStock、QueueRecord
控制类	**ConfirmBookController** 确认预约控制	1. 查询 PENDING_CONFIRM 待确认单据列表; 2. 校验单据存在且状态为 PENDING_CONFIRM; 3. 合法则置为 BOOKED 已预约; 4. 返回结果	CourseOrder
控制类	**SignController** 签到消课控制	1. 处理签到消课完整流程; 2. 查询 BOOKED 已预约单据; 3. 置为 COMPLETED 已完成; 4. 课程名额 +1; 5. 按提交时间筛选该课程待排队单据并激活为 PENDING_CONFIRM; 6. 排队记录置 HANDLED; 7. 提取会员信息并发起消息通知; 8. 返回结果	CourseOrder、CourseStock、QueueRecord、MessageNotifyUI
控制类	**ScheduleController** 排课控制	1. 查询全部未处理排队记录; 2. 执行新增排课、增加课程名额; 3. 按提交时间筛选该课程待排队单据并激活为 PENDING_CONFIRM; 4. 排队记录置 HANDLED; 5. 提取会员信息发起消息通知; 6. 返回结果	CourseStock、QueueRecord、CourseOrder、MessageNotifyUI
控制类	**CourseReportController** 报表控制	1. 接收时间区间; 2. 查询该区间 COMPLETED 已完成消课单据; 3. 调用 CourseStat 生成报表对象; 4. 返回报表或空数据提示;	CourseOrder、CourseStat

分类	类名	职责（知道什么 / 做什么）	协作者
		5. 支持定时器后台触发	
实体类	**CourseOrder** 预约 / 排队单据	1. 保存单据数据：orderId、courseId、memberInfo、bookCount、createTime、status; 2. 创建单据、设置状态; 3. 按状态查询（queryByStatus）、按 orderId 查询已预约（queryBookedOrder）; 4. 按排队记录 + 提交时间筛选待排队单据; 5. 批量设状态; 6. 从单据集合提取会员信息列表（extractMemberInfoList）	**无**（纯数据 / 自身查询，被各控制类调用，不调用别的类）
实体类	**CourseStock** 课程名额	1. 保存课程可预约名额 availableNum; 2. 查询课程名额; 3. 扣减名额（预约成功）; 4. 增加名额（排课新增、签到消课）	**无**
实体类	**QueueRecord** 排队记录	1. 保存排队记录：queueId、orderId、courseId、queueNum、status; 2. 创建排队记录; 3. 查询全部未处理记录、按课程查询; 4. 批量更新处理状态	**无**（通过 orderId 关联 CourseOrder 取会员信息，动作由控制类调 CourseOrder 完成，QueueRecord 自身不调用 CourseOrder）
实体类	**CourseStat** 上课统计报表	1. 存储统计结果（statId、timeRange、totalFinish）; 2. 根据已完成消课单据集合生成统计报表对象（generateStat）	**无**（只接收外部传入单据列表做计算存储，不调用任何类）

附录：plantuml 代码

用例图代码：

```
@startuml
left to right direction

actor 会员
actor 健身教练
actor 课程运营人员
actor 店长
actor MessageNotifyGateway as "消息通知网关\n(外部参与者)"

rectangle 线上健身房会员约课管理系统 {
  (提交课程预约请求)
  (确认课程预约)
  (办理签到消课)
  (新增课程排课)
  (生成上课统计报表)
  (发送会员消息通知) as "发送会员消息通知" <<extend>>
}

会员 --> (提交课程预约请求)
健身教练 --> (确认课程预约)
健身教练 --> (办理签到消课)
课程运营人员 --> (新增课程排课)
店长 --> (生成上课统计报表)
(发送会员消息通知) --> MessageNotifyGateway

'extend 扩展关系，虚线指向基用例，带扩展条件
(发送会员消息通知) -[dashed]-> (办理签到消课) : <<extend>> [存在待排队预约单据]
(发送会员消息通知) -[dashed]-> (新增课程排课) : <<extend>> [存在待排队预约单据]

note bottom of MessageNotifyGateway
外部消息推送网关，不启动任何用例
end note
@enduml
```

UC01 顺序图代码：

```
@startuml UC01 提交课程预约请求
actor Member as 会员
```

```
boundary CourseBookUI
control BookCourseController
entity CourseStock
entity CourseOrder
entity QueueRecord

会员-> CourseBookUI: submitBook(memberInfo, courseId, bookCount)
CourseBookUI -> BookCourseController: createCourseOrder(memberInfo, courseId, bookCount)

BookCourseController -> CourseOrder: new(memberInfo, courseId, bookCount)
CourseOrder --> BookCourseController: orderId

BookCourseController -> CourseStock: queryCourseStock(courseId)
CourseStock --> BookCourseController: availableNum

alt 课程名额充足 availableNum >= bookCount
    BookCourseController -> CourseOrder: setState(orderId, PENDING_CONFIRM)
    BookCourseController -> CourseStock: deductStock(courseId, bookCount)
    BookCourseController --> CourseBookUI: result(创建待确认单据，教练可确认预约, orderId)
else 课程名额已满 availableNum < bookCount
    BookCourseController -> CourseOrder: setState(orderId, PENDING_QUEUE)
    BookCourseController -> QueueRecord: new(orderId, courseId, queueNum)
    BookCourseController --> CourseBookUI: result(创建待排队预约单，已生成排队记录, orderId)
end

CourseBookUI --> 会员: show bookResult
@enduml
```

UC02 顺序图代码：

```
@startuml UC02 确认课程预约
actor Coach as 健身教练
boundary CoachUI
control ConfirmBookController
entity CourseOrder

健身教练-> CoachUI: queryPendingConfirm()
CoachUI -> ConfirmBookController: getPendingConfirmOrders()

ConfirmBookController -> CourseOrder: queryByStatus(PENDING_CONFIRM)
CourseOrder --> ConfirmBookController: pendingOrderList
ConfirmBookController --> CoachUI: pendingOrderList
CoachUI --> 健身教练: show OrderList(pendingOrderList)
```

```
健身教练-> CoachUI: confirmBook(orderId)
CoachUI -> ConfirmBookController: confirmBook(orderId)

alt 单据合法，状态 PENDING_CONFIRM
    ConfirmBookController -> CourseOrder: setState(orderId, BOOKED)
    ConfirmBookController --> CoachUI: result(预约确认完成)
else 单据不存在 / 状态不合法
    ConfirmBookController --> CoachUI: result(单据不存在或状态不允许确认预约)
end

CoachUI --> 健身教练: show confirmResult
@enduml


UC03 顺序图代码：
@startuml UC03 新增课程排课
actor Operator as 课程运营人员
actor MessageNotifyGateway as "消息通知网关"

boundary ScheduleUI
boundary MessageNotifyUI
control ScheduleController
entity QueueRecord
entity CourseStock
entity CourseOrder

课程运营人员-> ScheduleUI: queryQueueList()
ScheduleUI -> ScheduleController: getQueueList()

ScheduleController -> QueueRecord: queryAllUnfinished()
QueueRecord --> ScheduleController: queueList
ScheduleController --> ScheduleUI: queueList
ScheduleUI --> 课程运营人员: show queueList

课程运营人员-> ScheduleUI: courseScheduleAdd(courseId, addNum)
ScheduleUI -> ScheduleController: courseScheduleAdd(courseId, addNum)

ScheduleController -> CourseStock: addStock(courseId, addNum)
ScheduleController -> QueueRecord: queryByCourse(courseId)
QueueRecord --> ScheduleController: relatedQueueList
'按预约提交时间排序取待排队单据
ScheduleController -> CourseOrder: queryByQueueRecordsOrderByTime(relatedQueueList)
CourseOrder --> ScheduleController: queueWaitOrderList
```

```
alt 找到待排队预约单 queueWaitOrderList is not empty
    ScheduleController -> CourseOrder: batchSetState(orderIdList, PENDING_CONFIRM)
    ScheduleController -> QueueRecord: batchSetStatus(relatedQueueList, HANDLED)

    ScheduleController -> CourseOrder: extractMemberInfoList(queueWaitOrderList)
    CourseOrder --> ScheduleController: memberInfoList

    ScheduleController -> MessageNotifyUI: buildAndSendNotify(memberInfoList)
    MessageNotifyUI -> 消息通知网关: sendNotify(memberInfoList)
    消息通知网关 --> MessageNotifyUI: notifyResult
    MessageNotifyUI --> ScheduleController: notifyResult

    note over ScheduleController,MessageNotifyUI
        notifyResult=false 记录日志
        不回滚业务单据状态
    end note

    ScheduleController --> ScheduleUI: result(课程名额更新，排队单转为待确认，尝试通知会员)
else 无对应待排队预约单
    ScheduleController --> ScheduleUI: result(仅更新课程名额，无待处理排队单据)
end

ScheduleUI --> 课程运营人员: show result
@enduml
```

UC04 顺序图代码：

```
@startuml UC04 生成上课统计报表
actor ShopManager as 店长
boundary ReportViewUI
control CourseReportController
entity CourseOrder
entity CourseStat

note over CourseReportController:系统定时器可后台调用 buildCourseReport(timeRange),后台生成报表，无界面交互
店长-> ReportViewUI: requestReport(timeRange)
ReportViewUI -> CourseReportController: buildCourseReport(timeRange)

CourseReportController -> CourseOrder: queryOrders(timeRange, COMPLETED)
CourseOrder --> CourseReportController: orderList

alt orderList 不为空，存在已完成消课记录
```

```
    CourseReportController -> CourseStat: generateStat(orderList, timeRange)
    CourseStat --> CourseReportController: courseStatObj
    CourseReportController --> ReportViewUI: courseStatObj
    ReportViewUI --> 店长: show report
else orderList 为空，无上课记录
    CourseReportController --> ReportViewUI: result(暂无上课数据，报表为空)
    ReportViewUI --> 店长: show reportTip(msg)
end
@enduml

UC05 顺序图代码：
@startuml UC05 办理签到消课
actor Coach as 健身教练
actor MessageNotifyGateway as "消息通知网关"

boundary SignUI
boundary MessageNotifyUI
control SignController
entity CourseOrder
entity CourseStock
entity QueueRecord

健身教练-> SignUI: signFinish(orderId)
SignUI -> SignController: handleSign(orderId)

SignController -> CourseOrder: queryBookedOrder(orderId)
CourseOrder --> SignController: bookOrder

alt bookOrder 不为空
    SignController -> CourseOrder: setState(orderId, COMPLETED)
    SignController -> CourseStock: addStock(courseId,1)
    SignController -> QueueRecord: queryByCourse(courseId)
    QueueRecord --> SignController: relatedQueueList
    '按预约提交时间筛选排序待排队单据
    SignController -> CourseOrder: queryByQueueRecordsOrderByTime(relatedQueueList)
    CourseOrder --> SignController: queueWaitOrderList

    alt queueWaitOrderList is not empty
        SignController -> CourseOrder: batchSetState(orderIdList, PENDING_CONFIRM)
        SignController -> QueueRecord: batchSetStatus(relatedQueueList, HANDLED)

        SignController -> CourseOrder: extractMemberInfoList(queueWaitOrderList)
        CourseOrder --> SignController: memberInfoList
```

```
        SignController -> MessageNotifyUI: buildAndSendNotify(memberInfoList)
        MessageNotifyUI -> 消息通知网关: sendNotify(memberInfoList)
        消息通知网关 --> MessageNotifyUI: notifyResult
        MessageNotifyUI --> SignController: notifyResult

        note over SignController,MessageNotifyUI:通知失败记录日志，业务不回滚
        SignController --> SignUI: result(消课签到成功，排队记录置为已处理，已触发通知)
    else queueWaitOrderList is empty
        SignController --> SignUI: result(消课签到成功，无待处理排队单)
    end
else bookOrder 为空
    SignController --> SignUI: result(未找到该已预约单据)
end

SignUI --> 健身教练: show signResult
@enduml
```

类图代码：

```
@startuml 健身房约课系统_类图
'实体类
class CourseOrder {
- orderId: String
- courseId: String
- memberInfo: String
- bookCount: Integer
- createTime: String
- status: String
// PENDING_CONFIRM / PENDING_QUEUE / BOOKED / COMPLETED
+ new(memberInfo:String, courseId:String, bookCount:Integer)
+ setState(orderId:String, status:String)
+ queryByStatus(status:String): List<CourseOrder>
+ queryBookedOrder(orderId:String): CourseOrder
+ queryByQueueRecordsOrderByTime(resList:List): List<CourseOrder>
+ batchSetState(idList:List, status:String)
+ extractMemberInfoList(orderList:List): List
}

class CourseStock {
- courseId: String
- availableNum: Integer
```

```
+ queryCourseStock(courseId:String): Integer
+ deductStock(courseId:String, count:Integer)
+ addStock(courseId:String, addNum:Integer)
}

class QueueRecord {
- queueId: String
- orderId: String
- courseId: String
- queueNum: Integer
- status: String // UNHANDLED 未处理 / HANDLED 已处理
+ new(orderId:String, courseId:String, queueNum:Integer)
+ queryAllUnfinished(): List<QueueRecord>
+ queryByCourse(courseId:String): List<QueueRecord>
+ batchSetStatus(resList:List, status:String)
}

class CourseStat {
- statId: String
- timeRange: String
- totalFinish: Integer
+ generateStat(orderSet:List, timeRange:String): CourseStat
}

'边界类
class CourseBookUI {
+ submitBook(memberInfo:String, courseId:String, bookCount:Integer)
}

class CoachUI {
+ queryPendingConfirm()
+ confirmBook(orderId:String)
}

class SignUI {
+ signFinish(orderId:String)
}

class ScheduleUI {
+ queryQueueList()
+ courseScheduleAdd(courseId:String, addNum:Integer)
}
```

```
class ReportViewUI {
+ requestReport(timeRange:String)
}

class MessageNotifyUI {
+ buildAndSendNotify(memberInfoList:List): Boolean
}

class MessageNotifyGateway {
+ sendNotify(memberInfoList:List): Boolean
}

'控制类
class BookCourseController {
+ createCourseOrder(memberInfo:String, courseId:String, bookCount:Integer): String
}

class ConfirmBookController {
+ getPendingConfirmOrders(): List<CourseOrder>
+ confirmBook(orderId:String): String
}

class SignController {
+ handleSign(orderId:String): String
}

class ScheduleController {
+ getQueueList(): List<QueueRecord>
+ courseScheduleAdd(courseId:String, addNum:Integer):String
}

class CourseReportController {
+ buildCourseReport(timeRange:String): CourseStat
}

CourseBookUI ..> BookCourseController
BookCourseController ..> CourseOrder
BookCourseController ..> CourseStock
BookCourseController ..> QueueRecord

CoachUI ..> ConfirmBookController
ConfirmBookController ..> CourseOrder
```

```
SignUI ..> SignController
SignController ..> CourseOrder
SignController ..> CourseStock
SignController ..> QueueRecord
SignController ..> MessageNotifyUI

ScheduleUI ..> ScheduleController
ScheduleController ..> CourseStock
ScheduleController ..> QueueRecord
ScheduleController ..> CourseOrder
ScheduleController ..> MessageNotifyUI

MessageNotifyUI ..> MessageNotifyGateway

ReportViewUI ..> CourseReportController
CourseReportController ..> CourseOrder
CourseReportController ..> CourseStat

CourseOrder "1" -- "0..1" QueueRecord
@enduml
```

状态图代码：

```
@startuml 预约单据状态图
[*] --> Created : 创建 CourseOrder 单据对象
Created --> PENDING_CONFIRM : 课程名额充足，设置状态待确认
Created --> PENDING_QUEUE : 课程名额已满，设置待排队预约

PENDING_QUEUE --> PENDING_CONFIRM : 新增课程排课 / 签到消课，名额补充
PENDING_CONFIRM --> BOOKED : 健身教练确认预约
BOOKED --> COMPLETED : 健身教练办理签到消课

note right of COMPLETED
COMPLETED 是业务终态；
单据持久保存，用于统计报表，对象不销毁
end note
@enduml
```